

vector addition explanation

13 May 2026 01:49

HPC Assignment 6 — Parallel Vector Addition using OpenMP

This assignment performs:

Parallel Addition of Two Vectors

using:

- OpenMP
- parallel loops
- vector operations

1. What We Have To Do?

We have to:

1. input two vectors
2. add corresponding elements
3. perform addition in parallel using OpenMP
4. display resultant vector

Main Goal

To demonstrate:

data parallelism using OpenMP

2. What is Vector Addition?

Vector addition means:

adding corresponding elements of two vectors

Example

Vector A:

1 2 3

Vector B:

4 5 6

Result Vector C:

5 7 9

Because:

A	B	A+B
1	4	5
2	5	7
3	6	9

Formula

$C[i]=A[i]+B[i]$

Why Parallelism Suitable Here?

Because:

each addition is independent

Example:

$C[0] = A[0] + B[0]$

does NOT depend on:

$C[1]$

So multiple additions can happen simultaneously.

3. Overall Workflow

Input Vector A



Input Vector B



Parallel Addition



Store Result in Vector C



Display Result

4. Header Files

```
#include <iostream>
```

```
#include <omp.h>
```

Purpose

Header	Purpose
iostream	input/output
omp.h	OpenMP parallel programming

Namespace

```
using namespace std;
```

Avoids writing:

```
std::
```

repeatedly.

5. Main Function

```
int main()
```

Program execution starts here.

6. Input Vector Size

```
int n;
```

```
cout << "Enter size of vectors: ";
```

```
cin >> n;
```

User enters:

number of elements

Example

If:

n = 3

Then vectors contain:

3 elements

7. Dynamic Memory Allocation

```
int *A = new int[n];
```

```
int *B = new int[n];
```

```
int *C = new int[n];
```

Purpose

Creates dynamic arrays:

- A → first vector
- B → second vector
- C → result vector

Why Dynamic Allocation Used?

Because:

size entered at runtime

not fixed at compile time.

Memory Visualization

A → []

B → []

C → []

After input:

A → [1 2 3]

B → [4 5 6]

C → []

8. Input Vector A

```
for (int i = 0; i < n; i++)
```

```
    cin >> A[i];
```

Reads elements into vector A.

Input Vector B

```
for (int i = 0; i < n; i++)
```

```
    cin >> B[i];
```

Reads vector B.

9. MOST IMPORTANT PART — Parallel Addition

Code:

```
#pragma omp parallel for
```

```
MOST IMPORTANT LINE.
```

What Does It Do?

Converts loop into:

parallel loop

Different threads execute different iterations simultaneously.

Parallel Loop

```
for (int i = 0; i < n; i++) {  
    C[i] = A[i] + B[i];  
}
```

Each thread computes:

different vector elements

Example

Suppose:

A = [1 2 3 4]

B = [5 6 7 8]

Thread Distribution

Thread 1:

C[0] = 1+5

Thread 2:

C[1] = 2+6

Thread 3:

C[2] = 3+7

Thread 4:

C[3] = 4+8

All execute:

simultaneously

Why No Race Condition Here?

Because:

- each thread writes to different index
- no shared variable modification conflict

10. Output Result

```
for (int i = 0; i < n; i++)  
    cout << C[i] << " ";
```

Displays resultant vector.

Example Output

5 7 9

11. Memory Deallocation

```
delete[] A;  
delete[] B;  
delete[] C;
```

Why Needed?

Frees dynamically allocated memory.

Prevents:

memory leaks

What is Memory Leak?

Memory allocated but never released.
Can waste RAM.

12. Example Execution

Input:

A = [1 2 3]

B = [4 5 6]

Parallel Computation

Thread	Operation
--------	-----------

T1	1 + 4
----	-------

T2	2 + 5
----	-------

T3	3 + 6
----	-------

Output

5 7 9

13. Time Complexity

Sequential Vector Addition:
 $O(n)$

Parallel Vector Addition

Approximate:

$O\left(\frac{n}{P}\right)$

Where:

- P = processors/threads

Why Faster?

Because:

- multiple additions happen simultaneously

14. Advantages of Parallel Vector Addition

Advantage	Explanation
Faster execution	multicore utilization
Simple parallelization	operations independent
No synchronization needed	separate indices

15. Important Viva Questions

Q1. What is vector addition?

Addition of corresponding vector elements.

Q2. Why vector addition suitable for parallelism?

Because each operation is independent.

Q3. What does #pragma omp parallel for do?

Executes loop iterations simultaneously using multiple threads.

Q4. Why no race condition occurs here?

Because each thread writes to different array index.

Q5. Why dynamic memory allocation used?

Because vector size is known only at runtime.

Q6. What does delete[] do?

Frees dynamically allocated memory.

Q7. What is memory leak?

Allocated memory not released properly.

Q8. What is time complexity of vector addition?

Sequential:

$O(n)$

Parallel:

$O\left(\frac{n}{P}\right)$

16. Short Practical Explanation

This program performs parallel vector addition using OpenMP. Two vectors are added element-wise using a parallel loop with #pragma omp parallel for. Since each operation is independent, vector addition is highly suitable for parallel execution. Dynamic memory allocation is used for runtime-sized vectors, and memory is released using delete[].